

Robotics II – Section 1

4/2/2026

Haines - 662108625

LaBarge – 62026763

Pienaar – 662041924

Final Project Report

1. Introduction

Traditional mobile robotics often operate under the assumption that a goal is a fixed position in space. While this simplifies path planning, it fails to account for cases where targets are frequently in motion, such as on an assembly line. This report bridges the gap between classroom theory and practical application by exploring navigation strategies for dynamic goals. By moving beyond stationary endpoints, it is possible to develop more robust systems capable of functioning in fluid, real-world environments.

2. Problem Definition

The main objective of this project is to create an algorithm to construct a Moving Target Visibility Graph. Rather than focusing on the physical control of a robot, this work centers on the computational challenge of modeling a 2D environment where a target follows a known, predetermined path. In this deterministic framework, the "problem" is the successful generation of a time-augmented data structure that accounts for both static polygonal obstacles and the robot's constant velocity limits.

The technical scope is defined by two specific goals, the first of which is graph construction, which entails implementing the logic required to calculate nodes and edges.

The next step is evaluating the MTVG as a data structure by measuring how its complexity and construction time scale in comparison to traditional visibility graphs, particularly as obstacle density and trajectory discretization increase.

3. Related Literature

This project is grounded in the "Moving Target Visibility Graph" framework proposed by Bhat et al. The MTVG enhances traditional visibility graph theory by treating the target as a time-

parameterized trajectory rather than a fixed point. In this model, nodes represent feasible "meeting points" where the robot can realistically catch the target, while edges represent trajectories that satisfy both geometric clearance and temporal constraints. This framework essentially transforms a spatial navigation problem into a 4D synchronization task, ensuring the robot arrives at the right place at exactly the right time.

4. Approach

To implement the MTVG, an object-oriented programming approach was adopted in Python. Two classes were defined, an agent class and a map class. The agent class defines the target to intercept, as well as the agent which performs the interception. The map class holds data about the environment, as well as performs core MTVG construction logic.

The agent class is what is used to define the robot and its physical properties and states. This class will not only be used to define the agent, but also can be used to define the goal, as in this case, the goal is moving similarly to a holonomic robot with a predefined linear path.

This agent class works by initializing with the following parameters: position (x and y), velocity, and heading.

The only function in the agent class is `find_position(self,dt)` which takes in the velocity and heading of the robot, and outputs the position one timestep ahead of the current robot time. This is then updated in the robot class. The equations are the standard equations used in holonomic robot control

$$\vec{x}(k+1) = \vec{x}(k) + d\vec{x} \quad (1)$$

$$\vec{x}(k+1) = \vec{x}(k) + \begin{bmatrix} v_x \cos(\theta) \\ v_y \sin(\theta) \end{bmatrix} dt \quad (2)$$

As previously mentioned, this can be used to define both the robot and the goal it is tracking. The definition of this class was rather simple; the map class is much more complex.

The Map class encapsulates the environment representation and implements the core logic required to construct the Moving Target Visibility Graph (MTVG). It stores the spatial configuration of the environment, including obstacle geometry, the agent, and the moving target, and provides methods for visibility graph construction, visibility interval computation, and interception cost evaluation.

The environment is defined on a 2D grid, with a set of nodes:

$$V = \{(x_i, y_i)\} \quad (3)$$

consisting of the agent position and all obstacle vertices. Edges between nodes are constructed based on collision-free line-of-sight conditions

4.1 Algorithm

The core logic of the MTVG construction is given in Figure 4.1.1. It describes three distinct steps to creating a MTVG under our set of assumptions.

First, develop a static visibility graph, given in lines 1-12 of the algorithm.

Then, find the visibility intervals between each node and each target in the environment, given in lines 13 and 14. A visibility interval is the set of times where an edge can be drawn from a node in the graph to a particular target.

Finally, lines 15 through 19 instruct the calculation of edge costs for each node. The edge cost in this scenario was taken to be the minimum time of flight for an agent to intercept a target from a node. Although in other settings, the edge cost can assume different forms depending on the application, such as minimum fuel consumption, minimum jerk, etc.

Algorithm 1 Construct Moving Target Visibility Graph (MTVG)

Require: Start position p , target trajectory $\tau(t)$, obstacle set $\mathcal{O}$, max speed

```

 $V_{\max}$ 
Ensure: Moving Target Visibility Graph  $\tilde{G} = (\tilde{V}, \tilde{E})$ 
1: Extract all obstacle vertices  $V_{\mathcal{O}}$ 
2:  $V_{\text{vis}} \leftarrow V_{\mathcal{O}} \cup \{p\}$ 
3: Initialize  $E_{\text{vis}} \leftarrow \emptyset$ 
4: for each pair  $(u, w) \in V_{\text{vis}} \times V_{\text{vis}}$  do
5:   if line segment  $[u, w]$  lies entirely in free space then
6:     Add edge  $(u, w)$  to  $E_{\text{vis}}$ 
7:   end if
8: end for
9: Define moving target node  $s$ 
10:  $\tilde{V} \leftarrow V_{\text{vis}} \cup \{s\}$ 
11:  $\tilde{E} \leftarrow E_{\text{vis}}$ 
12: for each  $q \in V_{\text{vis}}$  do
13:   Compute visibility interval:

$$I_q = \{t \geq 0 \mid \text{line segment } [q, \tau(t)] \text{ is collision-free}\}$$

14:   if  $I_q \neq \emptyset$  then
15:     Compute edge cost  $c(q, s, t)$ :

$$c(q, s, t) = \min_{\Delta t \geq 0} \{ \Delta t \mid t + \Delta t \in I_q, \|\tau(t + \Delta t) - q\| \leq v_{p_{\max}} \cdot \Delta t \}$$

16:     Add structured edge  $(q, s, I_q, c(q, s, t))$  to  $\tilde{E}$ 
17:   end if
18: end for
19: return  $\tilde{G} = (\tilde{V}, \tilde{E})$ 

```

Figure 4.1.1: MTVG construction algorithm written by group

4.2 Static Visibility Graph Construction

Data making up the static visibility graph was held in two sets: a set containing nodes and a set containing edges. Nodes were stored as a tuple of x and y coordinates, and included the agent, the target and all obstacle vertices. Edges were stored as two-element tuples, where each element is a tuple of x and y coordinates.

To construct the static visibility graph, each node must consider every other node in the graph as a candidate edge. Then, that candidate edge must be compared to all edges making up all obstacles, to check for a collision.

To implement this, every node in the map was looped over. For each node, all other nodes were again looped over, and a line constructed between the two nodes under consideration. Then, each obstacle vertex was looped over, and another line was constructed between adjacent vertices of an obstacle. Given one line making up a candidate edge P_1P_2 , and another line making up a possible

colliding edge P_3P_4 , an intersection point could be calculated. To find the intersection point, first the edges were parameterized.

$$L_1(t) = P_1 + t(P_2 - P_1), t \in [0,1] \quad (4)$$

$$L_2(u) = P_3 + t(P_4 - P_3), u \in [0,1] \quad (5)$$

Then the parameters t and u can be solved for by setting $L_1 = L_2$.

$$t = \frac{(x_4 - x_3)(y_3 - y_1) - (y_4 - y_3)(x_3 - x_1)}{(x_4 - x_3)(y_2 - y_1) - (y_4 - y_3)(x_2 - x_1)} \quad (6)$$

$$u = \frac{(x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1)}{(x_4 - x_3)(y_2 - y_1) - (y_4 - y_3)(x_2 - x_1)} \quad (7)$$

If both t and u are within $[0, 1]$ and the denominators of equations 6 and 7 are not zero, then a collision exists.

Overall, the construction of the static visibility graph is an expensive operation of $O(N_n^3)$ time complexity, where N_n is the number of nodes in the map; as each node must consider each other node to construct a candidate edge, and then all adjacent nodes of all obstacles must be considered as a possible collision.

4.3 Visibility Interval Construction

Visibility intervals were stored as a dictionary in the map class, where the key is a node, and the value is a list of time intervals. Each time interval is stored as a two-element list, where the first element is the starting time of a visibility interval, and the second element is the ending time of a visibility interval. This approach allows for proper handling of visibility intervals for nodes which may see a target pass behind obstacles one or more times and therefore have multiple distinct visibility intervals.

To find the visibility intervals for each node, the trajectory of the target in a simulation was

discretized with respect to time. Then, for each time step within the simulation, edges were constructed between all nodes in the map and the target at its position in time. If a node did not see the target at the previous timestep, then a new visibility interval was opened, with the initial and final time in the interval being set to the current time. If a node did see the target in the previous timestep, then the final time in the last visibility interval was overwritten with the current time.

The cost of this operation is dependent on the number of nodes in the graph, the number of targets, and how finely time is discretized, and has a time complexity of $O(N_n N_t N_{ts})$ where N_t is the number of targets and N_{ts} is the number of timesteps in a simulation.

4.4 Edge Cost Calculation

Edge costs were stored as a dictionary in the map class, where the key is a node, and the value is a three-element tuple containing the minimum time of flight to intercept the target, the agent velocity, and the heading angle required to achieve that interception.

To compute the edge cost for a given node q , each visibility interval associated with that node was iterated over. For each interval, the following constrained optimization problem was solved:

$$\min_{\Delta t, v_a, \theta} \Delta t \quad (8)$$

Subject to

$$q + v_a \tan(\theta) \Delta t = \tau(\Delta t) \quad (9)$$

$$\Delta t \in [t_0, t_f], v_a \in [0, v_a], \theta \in [0, 2\pi] \quad (10)$$

Where $\tau(\Delta t)$ is the position of target at time Δt . $[t_0, t_f]$ is the visibility interval and v_a is the agent's maximum speed. The equality constraint enforces

that the agent, departing from node q , arrives at the same position as the target at time Δt . The problem was solved using the Sequential Least Squares Programming algorithm from the SciPy library, and the minimum cost found across all visibility intervals for a given node was retained as that node's edge cost.

Although this optimization problem may be non-trivial, for simple environments such as those considered in our testing, it can be assumed negligible for time complexity analysis. Taking this assumption, the time complexity of edge cost calculation is $O\left(\frac{N_n N_t N_{ts}}{2}\right)$. The factor of one half is introduced because in the worst case, the lowest edge cost for each node exists within the final visibility interval, and the maximum number of visibility intervals a node can have is $\frac{N_{ts}}{2}$. This occurs in a situation where a target alternates from being visible and hidden from a node at every timestep in a simulation.

5.5 Performance Experiments

To evaluate the computational cost of the construction of the MTVG, a series of performance experiments were conducted by varying the complexity of the environment and measuring execution time. Obstacle sets, each consisting of eight equilateral triangles of unit side length arranged in a vertical column, were introduced into a 40×20 environment in increasing quantity. The number of obstacle sets varied from one to ten, with each set placed equally along the x-axis between the agent, target, and other obstacle sets. For each configuration, execution time was recorded separately for the static visibility graph construction and the MTVG construction.

For each trial, the agent was placed at position (1,10) with a maximum speed of one, and the target was placed at position (39,19) moving at speed of one on a heading of 270° . The simulation was run with a final time $t_f = 200s$ and a time step of $\Delta t = 0.5 s$.

5. Results and Analysis

Following the approach outlined in Section 2, a method of constructing and evaluating an MTVG was completed successfully. Figure 5.1 visualizes an MTVG from an initial trial with hard coded obstacle, agent, and target parameters.

The edges of the static visibility graph are shown in red, the edges of interception trajectories from nodes with visibility intervals with the target are shown in yellow, and the interception trajectory with the lowest edge cost is shown in bold red. In this trial, the agent was placed at position (1, 1) with a maximum speed of one, and the target was placed at position (9, 7) moving at a speed of one and heading of 270° . Two triangular obstacles were placed in the environment. The optimal interception was found at a time of flight of $\Delta t = 2.04$ seconds, on a heading of $\theta = 18.9^\circ$

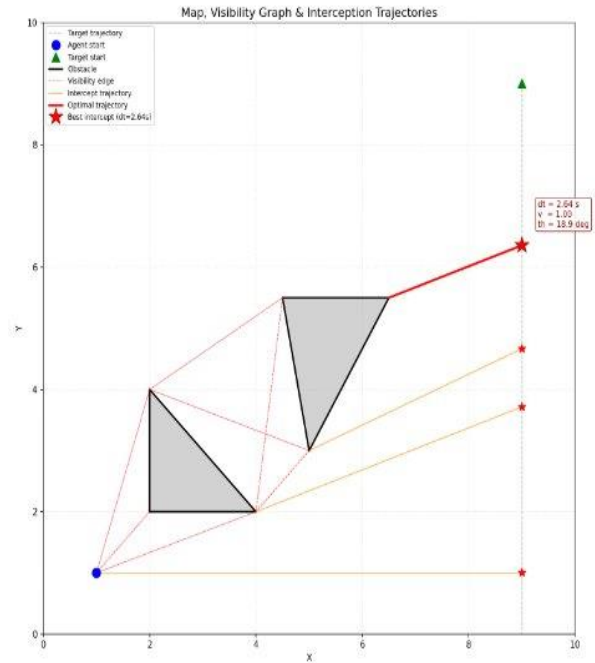


Figure 5.1: Initial MTVG results

The key details which confirm proper MTVG construction is first the existence of edges between all nodes without collisions, and the existence of edges between nodes and interception points on the target's trajectory.

Crucially, these edges are not necessarily perpendicular to the target's trajectory; showing that the minimum edge cost is given by the agent traveling at its maximum velocity toward the target rather than simply intercepting it at the closest point between the target and node.

Once proper MTVG construction was verified in the simple case above, the performance experiments were conducted. Visualizations of the MTVG in the case with one obstacle set and the case with 10 obstacle sets are given in Figure 5.2.

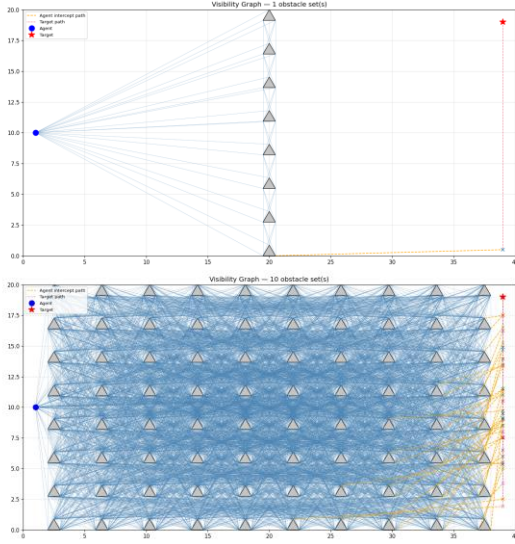


Figure 5.2: MTVG in the one obstacle set (top figure) and 10 obstacle set (bottom figure) cases.

The execution time results are shown in Figure 5.3.

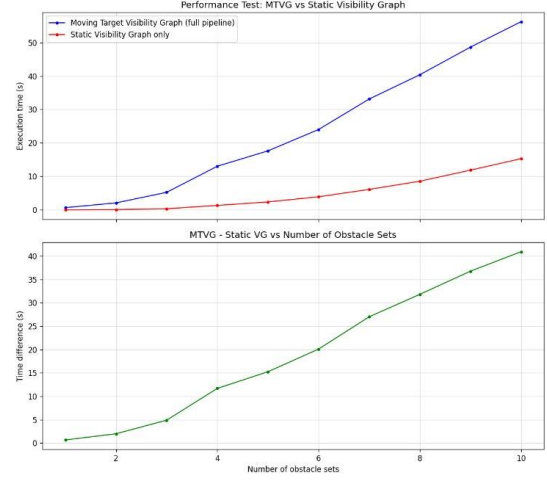


Figure 5.3: Execution time results from performance scaling experiments

The top subfigure displays the raw execution time vs obstacle sets for MTVG construction, in blue, and static visibility graph construction, in red. The bottom subfigure displays the difference between MTVG and static visibility graph construction execution time.

The top subfigure illustrates that the MTVG is considerably more expensive to construct than the static visibility graph for environments with up to 250 nodes. Additionally, execution time for both the MTVG and static visibility graph seem to be growing according to a cubic relationship with respect to obstacle sets. This is expected, since we deduced in that the time complexity of the MTVG is

$$O\left(N_n^3 + N_n N_t N_{ts} + \frac{N_s N_t N_{ts}}{2}\right) = O\left(N_n^3 + \frac{3}{2} N_n N_t N_{ts}\right) \quad (8)$$

While the time complexity of the static visibility graph is simply $O(N_n^3)$.

Subtracting the time complexity of the static visibility graph from the time complexity of the MTVG yields $O\left(\frac{3}{2} N_n N_t N_{ts}\right)$, a super linear first order relationship between compute and nodes in an environment, which aligns with our experimental results in the bottom subfigure of Figure 5.3.

These results both confirm our analysis of the cost of constructing the MTVG, as well as gives guidance for future work. The dominating operation in MTVG construction is the construction of the static visibility graph and therefore should be the first subject of optimization. Luckily, the operations which make up the construction of the visibility graph are all independent of each other, which makes it a promising subject for parallelization. In fact, *all* operations of the MTVG are parallelizable and low-cost calculations. In a sufficiently parallel implementation of the MTVG, the algorithms bottleneck would be the memory overheads of distributing the map among parallel processors, since each node requires full knowledge of the map in order to calculate its connected nodes, visibility intervals, and edge costs.

6. Discussion

The results show that incorporating time constraints into a visibility graph allows for the tracking of moving targets. The MTVG successfully extends the visibility graph made in class by accounting for the time dependent nature of the moving target, which allowed the program to both identify when and where a collision free path should exist.

The main limitation is computational cost. The static visibility graph is already expensive, and adding time discretization plus per-node optimization significantly increases runtime. Performance scales poorly as obstacle count and time resolution increases.

Time discretization is another weakness. Smaller time steps improve accuracy but greatly increase computation time, while larger steps risk missing valid interception windows.

The edge cost optimization adds further overhead and can be slow due to repeated nonlinear solves. It works for small problems but does not scale well.

Despite this, the method is modular and accurate for small environments. Most computations (visibility checks, interval construction, cost evaluation) are independent and could be parallelized.

Overall, MTVG is effective but computationally heavy, and scaling requires better algorithms or parallel implementation.

7. Future Steps

The future steps continuing from this project should focus on evolving the MTVG from a static model into a dynamic navigation tool. A primary goal is the integration of real-time replanning, allowing the system to adapt when target trajectories are uncertain or subject to change. To enhance computational efficiency, future iterations should move away from time discretization in favor of closed-form visibility intervals and replace general numerical optimization with precomputed analytic solutions. Finally, implementing parallelized graph construction will be essential for scaling the algorithm to accommodate more complex, high-density environments without sacrificing performance.

Future work could also focus on integrating full path planning over the constructed MTVG, allowing the robot to properly navigate the environment and create paths that would allow for real world implementation.

8. Team contributions

Haines	Introduction, Problem Definition, Related Literature, Future Steps
LaBarge	Approach, Discussion, Future steps
Pienaar	All coding and experimentation Approach section Results and Analysis section

9. References

[1] A. Bhat, G. Gutow, B. Vundurthy, Z. Ren, S. Rathinam, and H. Choset, "A Complete Algorithm for a Moving Target Traveling Salesman Problem with Obstacles," arXiv:2409.09852v3, October 2024.

[2] E. Yel, "Robotics II Lecture Slides," Rensselaer Polytechnic Institute, Robotics II, Spring 2026.